Lab 1 – Språkteknologi: program code

Name: Michail Pettas

File: lab1.py. Run with: python lab1.py (Python 3.11, spaCy 3.8).

```
"""
Lab 1 - Språkteknologi
spaCy: tokenization, POS tagging, dependency parsing, multilingual NLP.

Requirements:
    pip install spacy
    python -m spacy download en_core_web_sm
    python -m spacy download sv_core_news_sm
"""

from pathlib import Path

import spacy
from spacy import displacy
from spacy.tokens import Doc

nlp = spacy.load("en_core_web_sm")
OUT_DIR = Path(__file__).parent / "output"
OUT_DIR.mkdir(exist_ok=True)


def header(title):
    print("\n" + "=" * 78)
    print(title)
    print("=" * 78)


def token_table(doc, columns):
    """Print one row per token; `columns` is a list of (name, width, getter)."""
    print("  ".join(f"{name:<{width}}" for name, width, _ in columns))
    print("  ".join("-" * width for _, width, _ in columns))
    for token in doc:
        if token.is_space:
            continue
        print("  ".join(f"{str(get(token)):<{width}}" for _, width, get in columns))


# ---------------------------------------------------------------------------
# Task 1: Tokenization
# ---------------------------------------------------------------------------
TEXT_1 = (
    "On Jan. 15, 2024, Dr. Smith's research on AI-generated art sparked debates "
    "on 'true' creativity. While some critics call it mere imitation, others see "
    "it as a $10 million revolution in human–AI collaboration!"
)


def task1():
    header("TASK 1: Tokenization with spaCy")
    doc = nlp(TEXT_1)
    tokens = [token.text for token in doc]
    print(f"Number of tokens: {len(tokens)}\n")
    print(tokens)
    print()
    token_table(doc, [
        ("i", 3, lambda t: t.i),
```

```
        ("text", 12, lambda t: t.text),
        ("is_punct", 8, lambda t: t.is_punct),
        ("like_num", 8, lambda t: t.like_num),
        ("is_currency", 11, lambda t: t.is_currency),
        ("is_stop", 7, lambda t: t.is_stop),
        ("whitespace_after", 16, lambda t: repr(t.whitespace_)),
    ])
    return doc


# ---------------------------------------------------------------------------
# Task 2: Filtering tokens (no numbers, no number words, no stop words)
# ---------------------------------------------------------------------------
def task2(doc):
    header("TASK 2: Tokens that are not numbers, number words or stop words")
    kept, removed = [], []
    for token in doc:
        if token.like_num:
            removed.append((token.text, "like_num"))
        elif token.is_stop:
            removed.append((token.text, "is_stop"))
        else:
            kept.append(token.text)
    print(f"Kept ({len(kept)} tokens):")
    print(kept)
    print(f"\nRemoved ({len(removed)} tokens):")
    print(removed)

    # Stricter variant: also drop punctuation and currency symbols,
    # which leaves only content words.
    content = [t.text for t in doc
               if not (t.like_num or t.is_stop or t.is_punct or t.is_currency)]
    print(f"\nVariant that also removes punctuation/currency ({len(content)} tokens):")
    print(content)

    # like_num should also catch number words
    probe = nlp("one two first third ten hundred million 2024 15 3.5")
    print("\nlike_num check on number words:",
          [(t.text, t.like_num) for t in probe])


# ---------------------------------------------------------------------------
# Task 3: spaCy tokenizer vs. str.split()
# ---------------------------------------------------------------------------
TEXT_3 = (
    "Hello, world! Isn't this state-of-the-art? I'm sure it is. "
    "Don't you think so, Dr. Brown? We'll see: prices rose 3.5% in the U.S."
)


def task3():
    header("TASK 3: spaCy tokenizer vs. Python's split()")
    split_tokens = TEXT_3.split()
    spacy_tokens = [t.text for t in nlp(TEXT_3)]
    print(f"Text: {TEXT_3}\n")
    print(f"split() -> {len(split_tokens)} tokens:")
    print(split_tokens)
    print(f"\nspaCy   -> {len(spacy_tokens)} tokens:")
    print(spacy_tokens)

    print("\nHow each approach handles specific phenomena:")
    for example in ["world!", "so,", "Isn't", "I'm", "don't", "state-of-the-art",
                    "AI-generated", "Dr.", "U.S.", "3.5%", "$10"]:
        print(f"  {example:<18} split(): {example.split()!s:<22} "
              f"spaCy: {[t.text for t in nlp(example)]}")
```

```
    # tag the same text twice: once with spaCy's tokens, once with split() tokens
    sentence = "I'm sure it is. Don't you think so, Dr. Brown?"
    own = nlp(sentence)
    presplit = nlp(Doc(nlp.vocab, words=sentence.split()))
    print("\nTags with spaCy tokens:")
    print([(t.text, t.tag_) for t in own])
    print("Tags with split() tokens:")
    print([(t.text, t.tag_) for t in presplit])


# ---------------------------------------------------------------------------
# Task 4: UPOS vs. XPOS
# ---------------------------------------------------------------------------
TEXT_4 = (
    "The quick brown fox jumps over the lazy dog, but why does it always pick on "
    "that poor dog? She saw the man with binoculars. Time flies like an arrow; "
    "fruit flies like a banana."
)


def task4():
    header("TASK 4: UPOS (token.pos_) vs. XPOS (token.tag_)")
    doc = nlp(TEXT_4)
    for n, sent in enumerate(doc.sents, 1):
        print(f"\nSentence {n}: {sent.text}")
        token_table(sent, [
            ("text", 11, lambda t: t.text),
            ("UPOS", 6, lambda t: t.pos_),
            ("XPOS", 5, lambda t: t.tag_),
            ("XPOS meaning", 38, lambda t: (spacy.explain(t.tag_) or "")[:38]),
            ("dep", 9, lambda t: t.dep_),
            ("head", 10, lambda t: t.head.text),
        ])

    # Show that the same word form gets different tags depending on context.
    print("\nSame word form, different context:")
    for word in ["flies", "like", "saw", "that", "dog"]:
        hits = [(t.text, t.pos_, t.tag_, t.dep_) for t in doc if t.lower_ == word]
        print(f"  {word:<6} {hits}")

    # "with binoculars": which word does the parser attach the PP to?
    with_token = next(t for t in doc if t.text == "with")
    print(f"\nPP attachment: 'with' is attached to '{with_token.head.text}' "
          f"(dep={with_token.dep_})")

    displacy_svg(list(doc.sents)[-1].as_doc(), "task4_time_flies.svg")

    # compare with the medium model, if it is installed
    try:
        nlp_md = spacy.load("en_core_web_md")
    except OSError:
        return
    print("\nComparison en_core_web_sm vs. en_core_web_md on the ambiguous sentences:")
    for sentence in ["She saw the man with binoculars.",
                     "Time flies like an arrow; fruit flies like a banana."]:
        for name, model in [("sm", nlp), ("md", nlp_md)]:
            print(f"  {name}: " + " ".join(f"{t.text}/{t.pos_}/{t.tag_}->{t.head.text}"
                                           for t in model(sentence) if not t.is_punct))


# ---------------------------------------------------------------------------
# Task 5: Travel queries - past vs. present vs. future/intent
# ---------------------------------------------------------------------------
QUERIES = [
```

```
    "We flew to Rome 3 days ago. I still haven't received the bill, please send it ASAP.",
    "I have flown to Rome this morning and forgot my laptop on the plane. "
    "Can you please connect me to lost and found?",
    "I'm flying to Rome next week. Can you check flight availability?",
    "We need to fly to Rome. Can you check flights for next Tuesday?",
    "I will fly to Rome next week. Can you check the flights?",
    "They will fly to Vienna next Monday. Can you check the flights?",
]

INTENT_VERBS = {"need", "want", "plan", "intend", "hope", "wish"}


def future_time_expression(sent):
    """True if the sentence contains a future time reference (next week, tomorrow, ...)."""
    words = {t.lower_ for t in sent}
    return bool(words & {"next", "tomorrow", "tonight", "upcoming", "soon"})


def classify_fly(verb):
    """Classify the time reference of a form of 'fly' using POS, tags, morph and deps."""
    auxes = [c for c in verb.children if c.dep_ == "aux"]
    aux_lemmas = {a.lemma_ for a in auxes}
    tense = verb.morph.get("Tense")

    # will/shall + VB  ->  future
    if any(a.tag_ == "MD" and a.lemma_ in {"will", "shall"} for a in auxes):
        return "FUTURE", "modal aux 'will' (MD) + base form (VB)"
    # need/want/plan + to + VB  ->  intention (future)
    if verb.dep_ == "xcomp" and verb.head.lemma_ in INTENT_VERBS and "to" in aux_lemmas:
        return "FUTURE (intent)", (f"xcomp of '{verb.head.text}' ({verb.head.tag_}), "
                                   "'to' (TO/aux) + base form (VB)")
    # have + VBN  ->  present perfect: completed action
    if verb.tag_ == "VBN" and "have" in aux_lemmas:
        return "PAST (present perfect)", "aux 'have' + past participle (VBN), Tense=Past"
    # be + VBG  ->  present progressive; future only with a future time expression
    if verb.tag_ == "VBG" and "be" in aux_lemmas:
        if future_time_expression(verb.sent):
            return "FUTURE (planned)", ("aux 'be' (VBP) + VBG, Aspect=Prog, "
                                        "with future time expression ('next ...')")
        return "PRESENT (ongoing)", "aux 'be' + VBG, no future time expression"
    if verb.tag_ == "VBD" or "Past" in tense:
        return "PAST", "simple past (VBD), Tense=Past"
    return "PRESENT", f"tag {verb.tag_}"


def task5():
    header("TASK 5: Travel queries - which ones are about a planned future flight?")
    booking_queries = []
    for n, query in enumerate(QUERIES, 1):
        doc = nlp(query)
        print(f"\nQuery {n}: {query}")
        token_table(doc, [
            ("text", 12, lambda t: t.text),
            ("POS", 5, lambda t: t.pos_),
            ("TAG", 4, lambda t: t.tag_),
            ("dep", 8, lambda t: t.dep_),
            ("head", 10, lambda t: t.head.text),
            ("morph", 40, lambda t: t.morph),
        ])
        fly_verbs = [t for t in doc if t.lemma_ == "fly" and t.pos_ == "VERB"]
        for verb in fly_verbs:
            label, reason = classify_fly(verb)
            aux = " ".join(a.text for a in verb.children if a.dep_ == "aux")
            print(f"  -> '{(aux + ' ' + verb.text).strip()}': {label}  [{reason}]")
            if label.startswith("FUTURE"):
```

```
                booking_queries.append(n)
    print(f"\nQueries about a planned future flight (booking intent): {booking_queries}")


# ---------------------------------------------------------------------------
# Task 6: Dependency tree
# ---------------------------------------------------------------------------
TEXT_6 = "Although she was tired, she finished her work before midnight."


def task6():
    header("TASK 6: Dependency tree")
    doc = nlp(TEXT_6)
    token_table(doc, [
        ("i", 2, lambda t: t.i),
        ("text", 9, lambda t: t.text),
        ("POS", 5, lambda t: t.pos_),
        ("dep", 6, lambda t: t.dep_),
        ("explanation", 26, lambda t: spacy.explain(t.dep_) or ""),
        ("head", 9, lambda t: t.head.text),
        ("children", 30, lambda t: [c.text for c in t.children]),
    ])

    root = next(t for t in doc if t.dep_ == "ROOT")
    subj = [c for c in root.children if c.dep_ in ("nsubj", "nsubjpass")]
    obj = [c for c in root.children if c.dep_ in ("dobj", "obj")]
    print(f"\nMain clause verb (ROOT): {root.text}")
    print(f"Subject of main clause:  {[s.text for s in subj]}")
    print(f"Object of main clause:   {[' '.join(w.text for w in o.subtree) for o in obj]}")

    although = next(t for t in doc if t.lower_ == "although")
    clause_head = although.head
    print(f"\n'Although': dep={although.dep_} ({spacy.explain(although.dep_)}), "
          f"head='{clause_head.text}'")
    print(f"'{clause_head.text}': dep={clause_head.dep_} "
          f"({spacy.explain(clause_head.dep_)}), head='{clause_head.head.text}'")
    print("Subordinate clause:", " ".join(t.text for t in clause_head.subtree))

    print("\nTree (indented):")
    print_tree(root)
    displacy_svg(doc, "task6_dependency_tree.svg")


def print_tree(token, depth=0):
    print(f"{'    ' * depth}{token.text} [{token.dep_}, {token.pos_}]")
    for child in token.children:
        print_tree(child, depth + 1)


def displacy_svg(doc, filename):
    svg = displacy.render(doc, style="dep", jupyter=False,
                          options={"compact": False, "distance": 110})
    (OUT_DIR / filename).write_text(svg, encoding="utf-8")
    print(f"\nSaved dependency visualisation to {OUT_DIR / filename}")


# ---------------------------------------------------------------------------
# Task 7: Passive voice
# ---------------------------------------------------------------------------
TEXT_7 = """John bought a ticket for the concert yesterday.
Alice helped her friend with the homework.
The team completed the project on time.
The problem was solved quickly by Mark.
A new product was launched by the company last week.
Flowers were planted in the garden by Sarah.
```

```
The decision was made yesterday."""


def verb_tense(verb):
    """Tense of a (possibly passive) verb phrase, based on its auxiliaries."""
    auxes = [c for c in verb.children if c.dep_ in ("aux", "auxpass")]
    if any(a.lower_ in {"will", "shall"} for a in auxes):
        return "future"
    for a in auxes + [verb]:
        if a.tag_ == "VBD" or (a is not verb and "Past" in a.morph.get("Tense")):
            return "past"
        if a.tag_ in ("VBZ", "VBP"):
            return "present"
    return "past" if "Past" in verb.morph.get("Tense") else "present"


def task7():
    header("TASK 7: Passive voice")
    for line in TEXT_7.splitlines():
        doc = nlp(line)
        sent = list(doc.sents)[0]
        root = sent.root
        passive_subj = [c for c in root.children if c.dep_ == "nsubjpass"]
        auxpass = [c for c in root.children if c.dep_ == "auxpass"]
        print(f"\n{line}")
        print("  deps:", [(t.text, t.dep_, t.tag_) for t in doc if not t.is_punct])
        if not (passive_subj and auxpass):
            print("  -> ACTIVE voice (no nsubjpass/auxpass)")
            continue
        patient = " ".join(t.text for t in passive_subj[0].subtree)
        agent_prep = [c for c in root.children if c.dep_ == "agent"]
        if agent_prep:
            agent = " ".join(t.text for t in agent_prep[0].subtree if t.dep_ != "agent")
        else:
            agent = "(not expressed - implicit agent)"
        print(f"  -> PASSIVE voice: auxpass='{auxpass[0].text}' ({auxpass[0].tag_}) "
              f"+ '{root.text}' ({root.tag_})")
        print(f"     patient (nsubjpass): {patient}")
        print(f"     agent   (agent->pobj): {agent}")
        print(f"     tense: {verb_tense(root)}  (aux morph: {auxpass[0].morph})")


# ---------------------------------------------------------------------------
# Task 8: Alisa flying from Rome
# ---------------------------------------------------------------------------
TEXT_8 = """Alisa flew from Rome to Vienna three days ago.
My parents have flown from Athens to Rome this morning to visit Alisa.
Alisa and Luk will fly from Rome to Paris next week for a conference.
They are flying from New York to Rome tomorrow.
John had flown from Rome to London before Alisa received the urgent call."""


def subjects_of(verb):
    """Subjects of a verb, including coordinated subjects (Alisa and Luk)."""
    subjects = []
    for child in verb.children:
        if child.dep_ in ("nsubj", "nsubjpass"):
            subjects.append(child)
            subjects.extend(child.conjuncts)
    return subjects


def origin_of(verb):
    """Object of a 'from' preposition attached to the verb."""
    for child in verb.children:
```

```
        if child.dep_ == "prep" and child.lower_ == "from":
            for pobj in child.children:
                if pobj.dep_ == "pobj":
                    return " ".join(t.text for t in pobj.subtree)
    return None


def task8():
    header("TASK 8: Sentences where Alisa is the subject of flying from Rome")
    matches = []
    for n, line in enumerate(TEXT_8.splitlines(), 1):
        doc = nlp(line)
        print(f"\n{n}. {line}")
        for verb in (t for t in doc if t.lemma_ == "fly" and t.pos_ == "VERB"):
            subjects = subjects_of(verb)
            origin = origin_of(verb)
            aux = [a.text for a in verb.children if a.dep_ == "aux"]
            print(f"  verb: {' '.join(aux + [verb.text])} (POS={verb.pos_}, TAG={verb.tag_}, "
                  f"morph={verb.morph})")
            print(f"  subjects: {[(s.text, s.dep_, s.pos_) for s in subjects]}")
            print(f"  from:     {origin}")
            ok = any(s.text == "Alisa" for s in subjects) and origin == "Rome"
            print(f"  -> {'MATCH' if ok else 'no match'}")
            if ok:
                matches.append(n)
        # Where does Alisa occur and in which role?
        for t in doc:
            if t.text == "Alisa":
                print(f"  'Alisa': POS={t.pos_}, TAG={t.tag_}, dep={t.dep_}, head='{t.head.text}'")
    print(f"\nMatching sentences: {matches}")


# ---------------------------------------------------------------------------
# Task 9: Multilingual NLP (Swedish)
# ---------------------------------------------------------------------------
TEXT_9 = (
    "Umeå universitet grundades 1965 och är i dag ett av Sveriges största lärosäten. "
    "Många studenter flyttar till Umeå för att studera språkteknologi, "
    "trots att vintrarna är långa och kalla."
)


def task9():
    header("TASK 9: Multilingual NLP - Swedish (sv_core_news_sm)")
    nlp_sv = spacy.load("sv_core_news_sm")
    doc = nlp_sv(TEXT_9)
    print(f"Text: {TEXT_9}\n")
    print("1) Tokens:")
    print([t.text for t in doc])
    print("\n2) POS tagging and 3) dependency parsing:")
    token_table(doc, [
        ("text", 13, lambda t: t.text),
        ("lemma", 12, lambda t: t.lemma_),
        ("UPOS", 6, lambda t: t.pos_),
        ("XPOS (SUC)", 22, lambda t: t.tag_),
        ("dep", 11, lambda t: t.dep_),
        ("head", 12, lambda t: t.head.text),
        ("morph", 45, lambda t: t.morph),
    ])
    for n, sent in enumerate(doc.sents, 1):
        root = sent.root
        subj = [t.text for t in root.children if t.dep_.startswith("nsubj")]
        print(f"\nSentence {n}: ROOT='{root.text}' ({root.pos_}), subject={subj}")
    print("\nNamed entities:", [(e.text, e.label_) for e in doc.ents])
    displacy_svg(list(doc.sents)[0].as_doc(), "task9_swedish_sentence1.svg")
```

```
    displacy_svg(list(doc.sents)[1].as_doc(), "task9_swedish_sentence2.svg")


if __name__ == "__main__":
    doc1 = task1()
    task2(doc1)
    task3()
    task4()
    task5()
    task6()
    task7()
    task8()
    task9()
```